Tests de Validation - Phase 1 & 2

Lecture du Format ELF ARM Big Endian

1 Organisation des tests

L'ensemble des scripts de test est regroupé dans un dossier dédié `TestsAutomatisées/`, facilitant ainsi leur gestion et leur exécution.

1.1 Structure du dossier de tests

Le dossier `TestsAutomatisées/` contient les scripts suivants :

Script	**Description**
`test_phase1.sh`	Validation complète de la Phase 1 (lecture ELF)
`test_E6.sh`	Test de l'étape E6 (fusion des sections)
`test_E7.sh`	Test de l'étape E7 (fusion des symboles)
`test_E8.sh`	Test de l'étape E8 (fusion des relocations)
`test_E9.sh`	Test structurel de l'étape E9 (cohérence ELF)
`test_E9_exec.sh`	Test fonctionnel de l'étape E9 (exécution sous QEMU)

Table 1 – Scripts de test disponibles

2 Synthèse des tests – Phase 1

2.1 Principe général de validation

La validation de la Phase 1 repose sur une comparaison systématique entre la sortie de notre programme et celle de l'outil de référence `arm-none-eabi-readelf`.

Pour chaque test :

1. Les deux programmes sont exécutés avec les mêmes options.
2. Les sorties sont normalisées afin de rendre les formats comparables.
3. Les sorties normalisées sont comparées à l'aide de l'outil `diff`.

Un test est validé lorsqu'aucune différence n'est détectée après normalisation.

2.2 Script de test automatisé

Le script `TestsAutomatisées/test_phase1.sh` automatise l'ensemble du processus de validation pour toutes les étapes de la Phase 1.

2.2.1 Options du script

Option	Description
`-d dossier`	Spécifie le dossier contenant les fichiers à tester (défaut : `./Examples_fusion`)
`-o option`	Option à tester : `-t`, `-S`, `-s`, `-r`, `-x` (défaut : `-t`)
`-a arg`	Argument supplémentaire pour l'option `-x` (nom ou index de section)
`-h`	Affiche l'aide

TABLE 2 – Options disponibles du script de test Phase 1

2.3 Étapes de validation

2.3.1 Étape A – En-tête ELF (`-t`)

Cette étape vérifie l'affichage des informations principales de l'en-tête ELF.
Normalisation appliquée : extraction des 19 dernières lignes de la sortie.
Critère de réussite : sortie identique à `readelf -h` après normalisation.

2.3.2 Étape B – Table des sections (`-S`)

Cette étape vérifie l'affichage de la table des sections et de leurs attributs.
Normalisation appliquée : Filtrage des lignes commençant (en-têtes de sections) et des espaces multiples
Critère de réussite : correspondance exacte avec la sortie de `readelf -S`.

2.3.3 Étape C – Contenu d'une section (`-x`)

Cette étape vérifie l'affichage du contenu binaire d'une section sous forme de dump hexadécimal.
Normalisation appliquée : Une fonction dédiée `normalize_x()` est utilisée pour extraire uniquement L'adresse memoire les données Hex (sans rep ascii). Cela permet d'éliminer les différences de formatage entre notre programme et `readelf`.
Critère de réussite : contenu hexadécimal identique à celui de `readelf -x` après normalisation.

2.3.4 Étape D – Table des symboles (`-s`)

Cette étape vérifie l'affichage des informations associées aux symboles.
Normalisation appliquée : Filtrage des lignes contenant des indices numériques et des espaces multiples
Critère de réussite : correspondance complète avec la sortie de `readelf -s`.

2.3.5 Étape E – Tables de relocation (`-r`)

Cette étape vérifie l'affichage des relocations (offset, type et symbole). Notre programme affiche directement l'index du symbole, tandis que `readelf` encode le symbole et le type dans le champ `r_info`. Une normalisation spécifique est donc appliquée en calculant :

$$\text{symbole} = \left\lfloor \frac{\text{Info}}{256} \right\rfloor$$

Normalisation appliquée : Calcul de l'index du symbole depuis le champ et Normalisation des espaces
Critère de réussite : égalité des triplets *(offset, type, symbole)* après normalisation.

2.4 Conclusion

La Phase 1 est validée à l'aide d'une procédure de test automatisée et reproductible, reposant sur une comparaison systématique avec l'outil de référence `arm-none-eabi-readelf`. L'ensemble des étapes A à E est validé sans divergence.

Le script teste uniquement les fichiers objets (`*.o`) présents dans le dossier spécifié, garantissant ainsi la cohérence des tests.

3 Validation – Phase 2 : Étapes E6 à E9

3.1 Principe général de validation

Objectif : valider progressivement la construction d'un fichier ELF relogeable correct, puis vérifier sa conformité après édition de liens et, lorsque nécessaire, après exécution.

Chaque étape est testée à l'aide d'un script dédié et comparée à une référence GNU (`arm-none-eabi-ld`), après normalisation si nécessaire, puis comparaison avec `diff`. La Phase 2 est validée lorsque l'ensemble des étapes E6 à E9 s'exécutent sans erreur.

3.2 Étape E6 – Fusion des sections

Objectif : produire `out6.o` en fusionnant les sections des deux fichiers d'entrée.
Marche à suivre :

- `cd TestsAutomatisées`
- `./test_E6 -f <fileA.o> <fileB.o> <out6.o>`

Le script construit l'union des *sections utiles* des deux entrées (en ignorant `.debug*` et `.rel*`/`.rela*`), puis applique des règles de validation *par section* sur `out6.o` :

- `PROGBITS + PROGBITS` : concaténation (A‖B), taille et contenu vérifiés ;
- `NOBITS + NOBITS` : somme des tailles ;
- autres types : conservation de la version A (ou copie si section unique) ;
- cas particulier : `.shstrtab` reconstruite (présence vérifiée).

Critère de réussite : `out6.o` est généré sans erreur, `readelf -S` ne produit aucun `Warning`, et toutes les sections respectent les règles de fusion ci-dessus.

3.3 Étape E7 – Fusion des symboles

Objectif : construire une table des symboles unifiée correcte dans `out7.o`.
Marche à suivre :

- `cd TestsAutomatisées`
- `./test_E7.sh <fileA.o> <fileB.o> <out7.o>`

Le script compare les symboles globaux de `out7.o` à une référence obtenue par `arm-none-eabi-ld -r` après normalisation (tri, filtrage de symboles automatiques).
Critère de réussite : la table des symboles globeaux de `out7.o` correspond à celle générée par `ld -r` après normalisation.

3.4 Étape E8 – Fusion des relocations

Objectif : fusionner les relocations dans `out8.o` en conservant la cohérence des offsets et des références de symboles.
Marche à suivre :

- `cd TestsAutomatisées`

— `./test_E8 <fileA.o> <fileB.o> <out8.o>`

Le test vérifie :

— le nombre total de relocations dans `.rel.text`, `.rel.data`, `.rel.bss` ;

— la correction des offsets pour les relocations du second fichier ;

— la cohérence des références de symboles après renumérotation ;

— la présence des sections `.rel.*`.

Critère de réussite : `out8.o` contient les relocations attendues, avec offsets corrigés, références cohérentes, et reste lisible par `readelf` sans erreur.

3.5 Étape E9 – Génération du fichier final

Objectif : produire `out9.o` exploitable pour l'édition de liens finale et vérifier sa conformité structurelle et fonctionnelle.

3.5.1 Tests structurels sur `out9.o`

3.5.2 Tests d'exécution après édition de liens

Marche à suivre :

— `cd TestsAutomatisées`

— `./test_E9.sh <A.o> <B.o>` (génère `out6.o out7.o out8.o out9.o` et `ref9.o`)

— `./test_E9_exec.sh out9.o ref9.o`

Le script d'exécution effectue l'édition de liens finale à l'adresse `0x10000` afin de produire `out.elf` (depuis `out9.o`) et `ref.elf` (depuis `ref9.o`). Les deux exécutables sont ensuite lancés sous `qemu-armeb`, la sortie est capturée puis comparée avec `diff`.
Critère de réussite : l'exécutable issu de `out9.o` ne provoque aucun crash et produit exactement la même sortie que la référence GNU.

3.6 Conclusion

Les étapes E6 à E9 ont été validées par des scripts automatisés et reproductibles. La validation structurelle garantit la conformité ELF, et la validation fonctionnelle confirme le bon comportement après édition de liens et exécution. La Phase 2 est ainsi considérée comme validée.